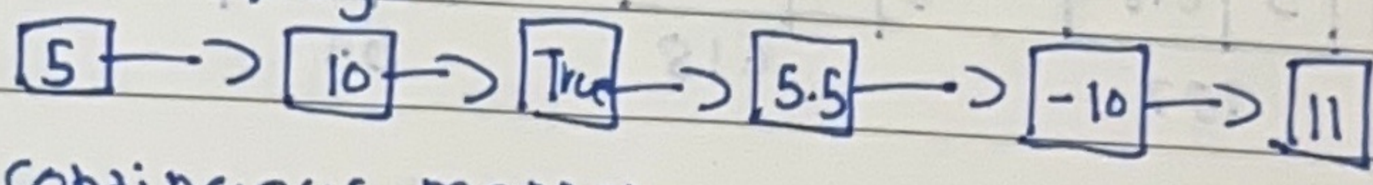
Introduction to linked lists in python.

1] what is linked list?.

```
                  0     1     2      3     4     5
Any_List = [5, 10, True, 5.5, -10, 11]
```

#How linked list displays



#Stores in continuous manner.

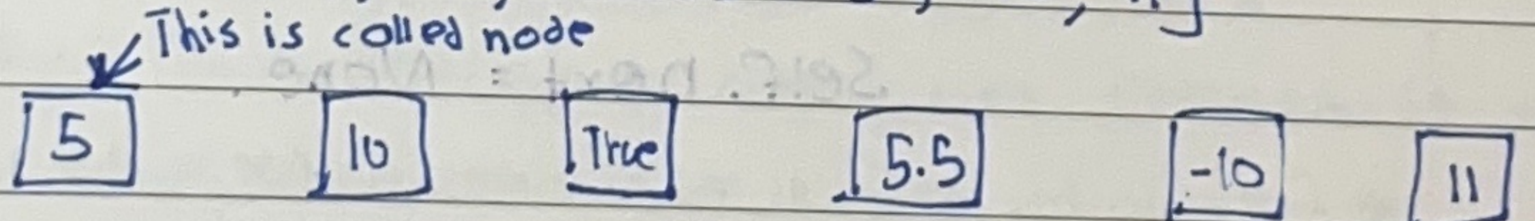
in Array How stored:

		5 [0]	10 [1]
True [2]	5.5 [3]	-10 [4]	11 [5]

This is How list is stored.

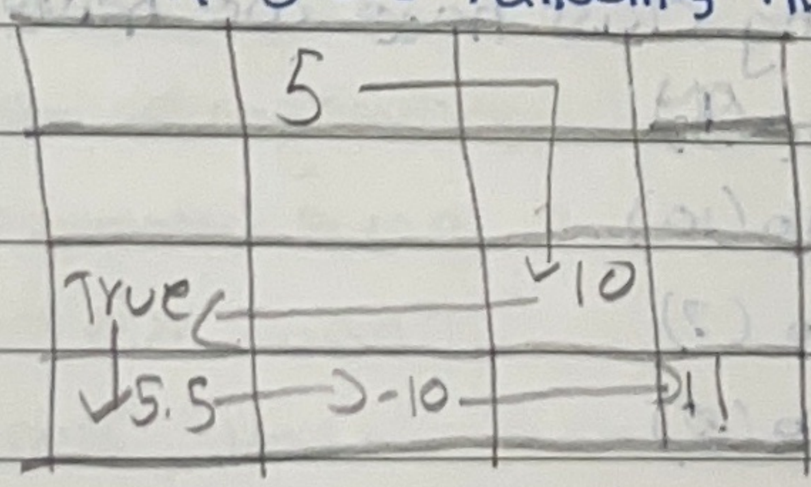
How we stored this in the linke list.

Any_List = [5, 10, True, 5.5, -10, 11]

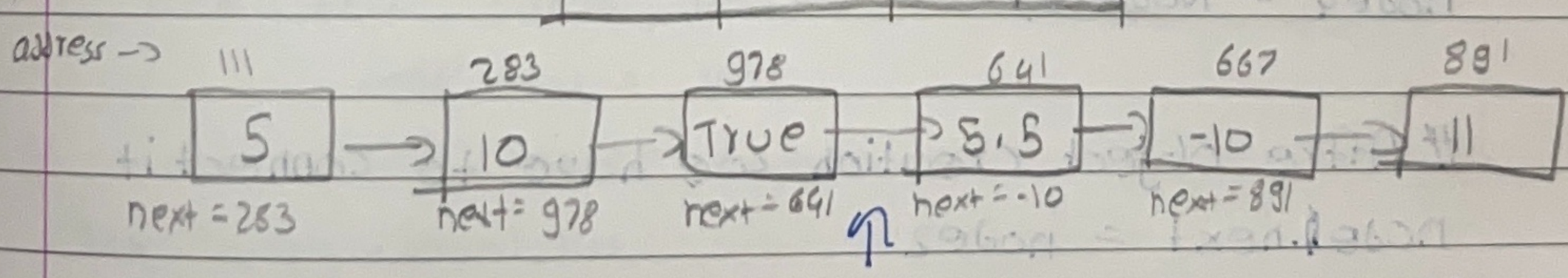


Ho these stores in memory

- it store randomly in memory ~~locatio~~.



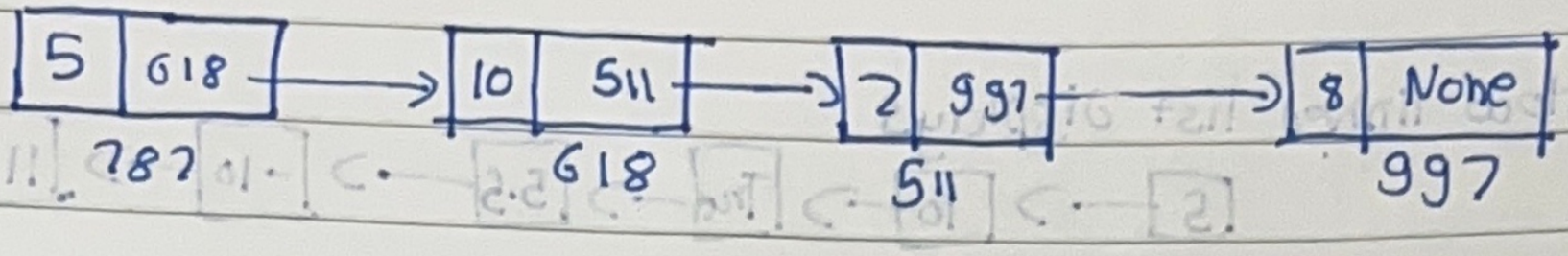
This is How linked list work



This is How a Linked List. stores in memory

- Node stores 2 values
  - Data (which data he has)
  - next (on which next address we have to go)

How To CREATE A NODE ADD SLL



- Singly Linked List - we can go only in one direction is called SLL
  - we can't go to previous ~~list~~ node.

How to write the code for this.

// first we have to create a class namely node.

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None  // At first node theres
# for now by default everyone's value don't have next address
  is none.                So by default is store none.
// After creating class node we have to create objects.
node1 = Node(5)    # everyone's address or value is
node2 = Node(10)     none because for now we don't
node3 = Node(7)      know is ka kya next hee
node4 = Node(8)

# After object creation we have to connect it
node1.next = node2
node2.next = node3
node3.next = node4
~~node4.next =~~
Print(node1) # we will thought it will display the value
        of node1 but it displays the address x00,19Ac
```

// Then how to print that value.

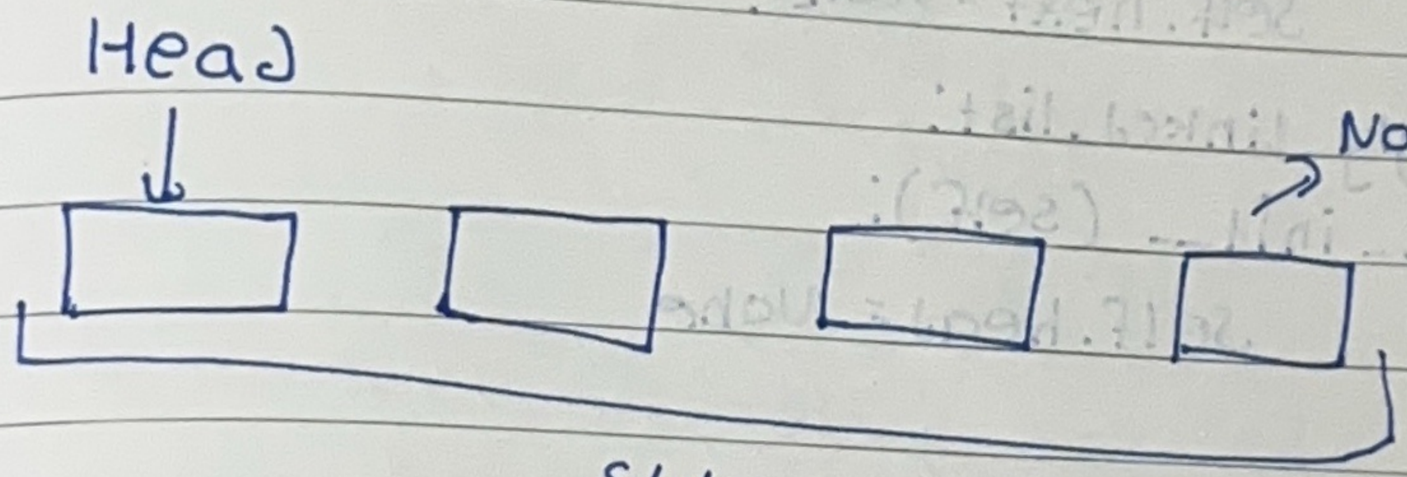
print(node1.val) # o/p:- 5.

✗ Print (node1.next) # it will display address again

✓ Print (node1.next.val) # o/p :- 10

✓ Print (node1.next.next.next.val) # o/p :- 8.

- How to create a SLL Class.
  - we have already created a class to create a nodes



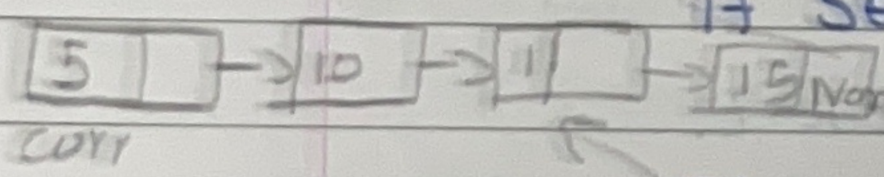
- The starting node will always be a Head.
- we have to store the head somewhere
- Head itself is not a node, we just create it to point ~~the~~ to the first node.
- Why we have to point first node?
  -> if we don't know what is the first node, then How will we traverse

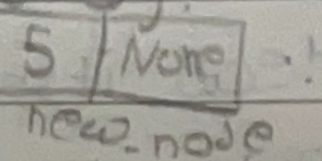
```
Class Node:
    def __init__(self, val):
        self.val = val
        self.next = none.
class Singly-Linked_list:
    def--init-(self):
        self.head = None. // At starting all nodes are by default null
```

APPEND.

- to add the node at the end
- There are 2 cases 1] SLL can be empty (what if)
  2] SLL is not empty.

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None
class Singly_linked_list:
    def __init__(self):
        self.head = None
    def append(self, val): # takes the value that what u want to print
        # we have created the attribute namely append
        # eg append(5)
        new_node = Node(val)
        if self.head == None: # if SLL is empty
            self.head = new_node
        else: # there are nodes in the SLL
            curr = self.head
            while curr.next is not None:
                curr = curr.next
            curr.next = new_node
```

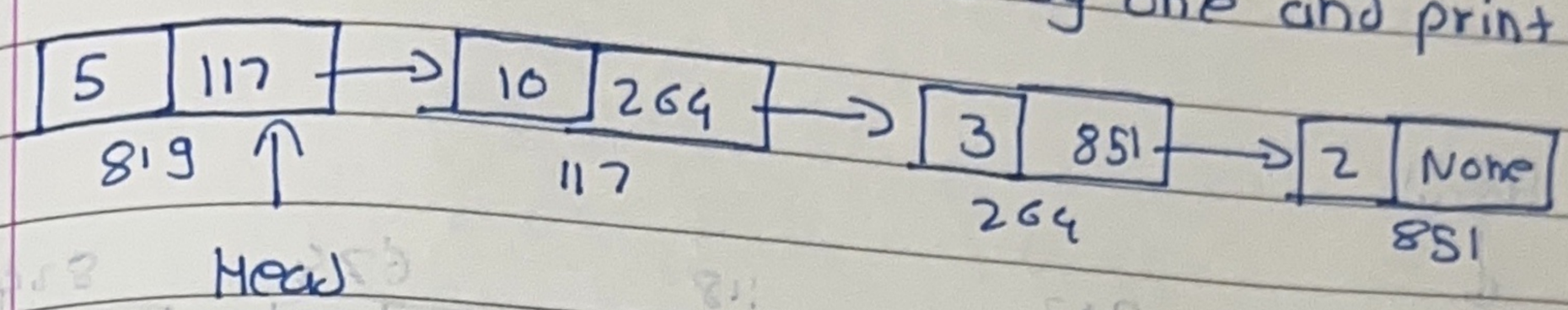




Time complexity
Space complexity

Traversal

itterate the value one by one and print there value.




```
def traversal(self):
    if self.head == None:
        Print("SLL is empty")
    else:
        curr = self.head .// current was first start with head
        while curr is not None:
            Print(curr.val, end=" ")
            curr = curr.next.
        Print()
```

- How all these will run.

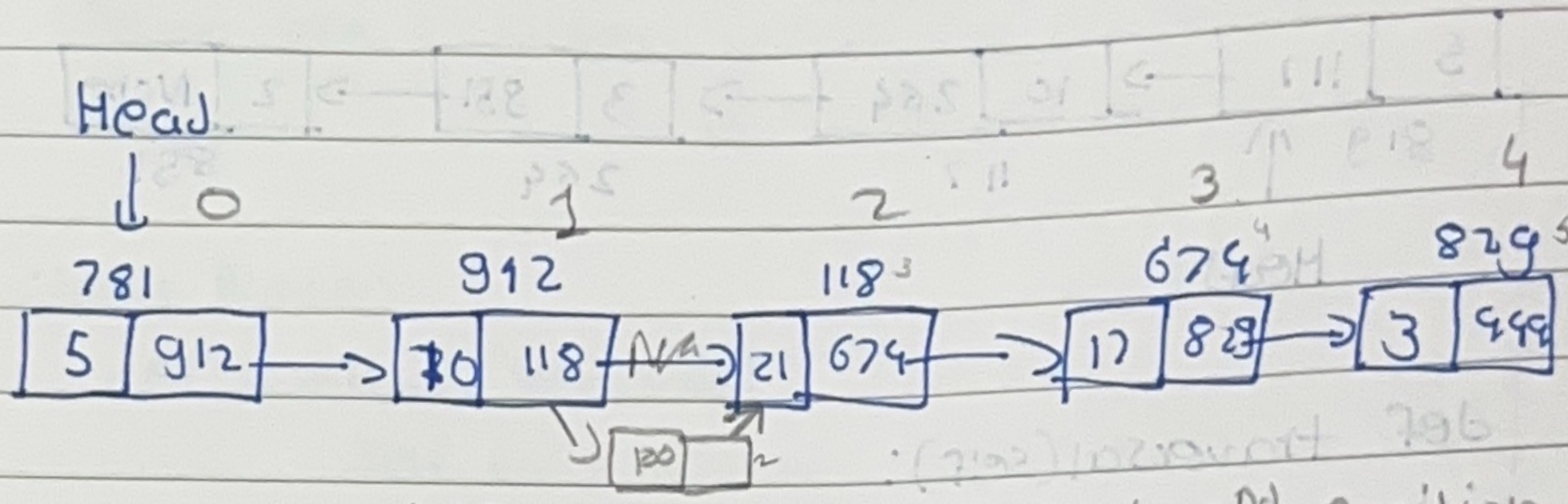
SLL is empty
```
SLL = SinglyLinkedList()  // create an object
SLL.traverse()
```

```
SLL = SinglyLinkedList()
SLL.append(10)
SLL.append(20)
SLL.append(30)
SLL.append(40)
SLL.append(1)
SLL.traverse()
```

O/P :- 10, 20, 30, 40, 1

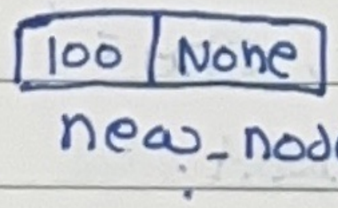
- INSERT At Specific Position.

this is also helpful in traversal.

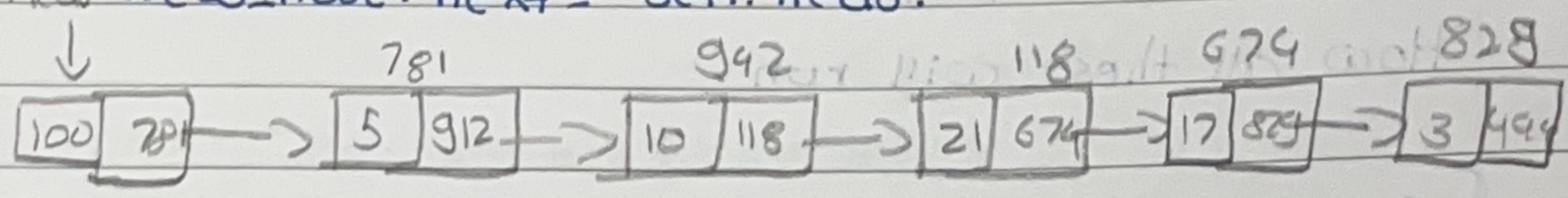


for eg.

i) 0, 100 (store 100 on 0th position)

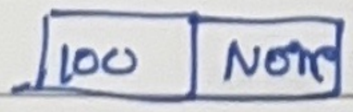


new_node.next = self.head.

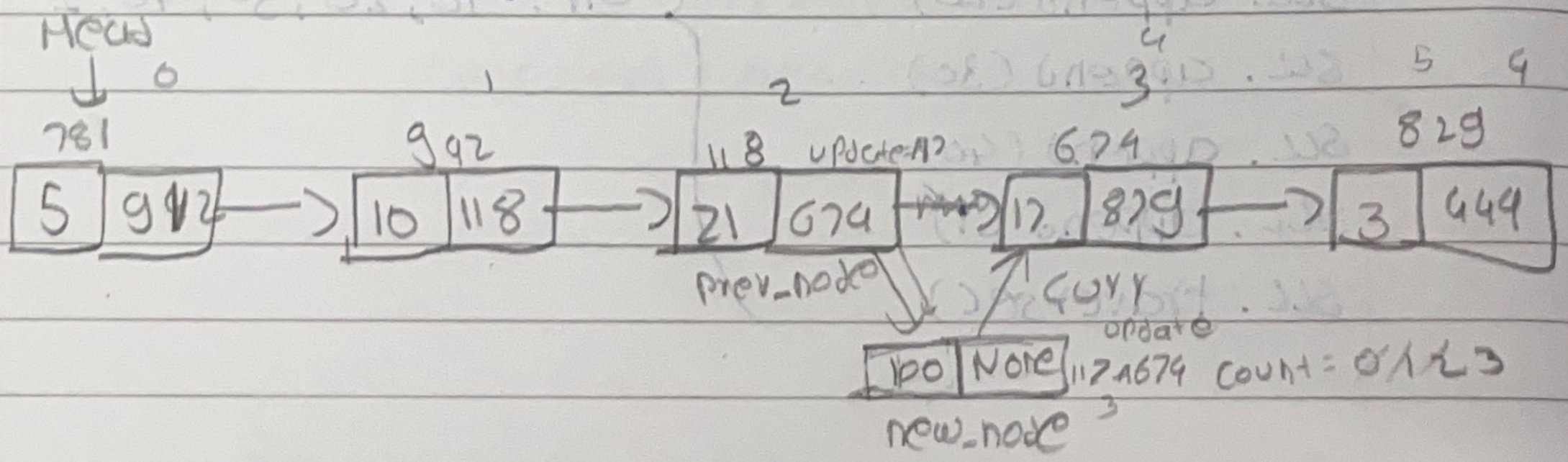


self.head = new_node.

ii) 3, 100



Count = 0

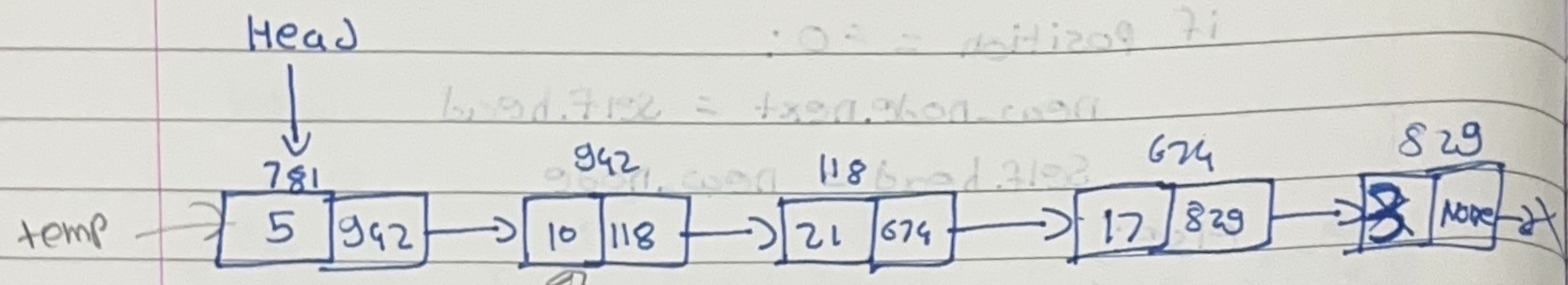


prev_node.next = new_node

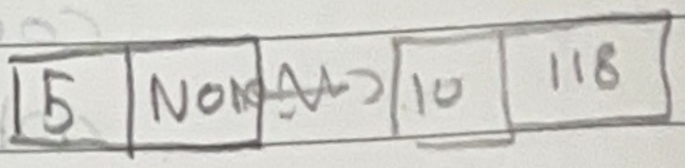
new_node.next = curr

```
def insert_at(self, val, position):
    new_node = Node(val)
    if position == 0:
        new_node.next = self.head
        self.head = new_node
    else:
        current = self.head
        prev_node = None
        count = 0
        while current != None and count< Position:
            prev_node = current
            current = current.next
            count += 1   count = count + 1
        prev_node.next = new_node
        new_node.next = current
```

- DELETE case1:- Delete Head node
  case2:- 17 node delete.

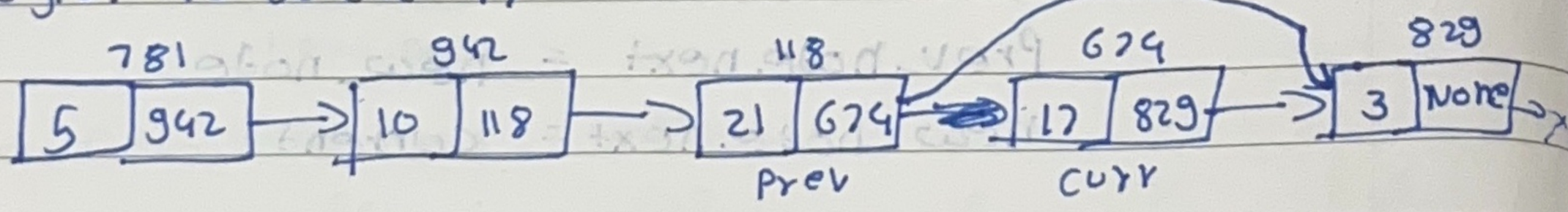


temp = self.head (temp = temporary)
self.head = self.head.next
temp.next = None // optional



del temp // optional

eg.2. to delete 17



Prev.next = curr.next
del curr // optional

```
#code
def delete(self, val):
    (curr) temp = self.head
    if temp.next != None:
        if temp.val == val:
            self.head = temp.next
            return
        else:
            found = false
            prev = None
            while temp != None:
                if temp.val == val:
                    found = True
                    break
                prev = temp
                temp = temp.next
```